

PRACTICAL RECORD

FastAPI Student Management System

Name: S Svara Haasini

Roll Number: 2520090170

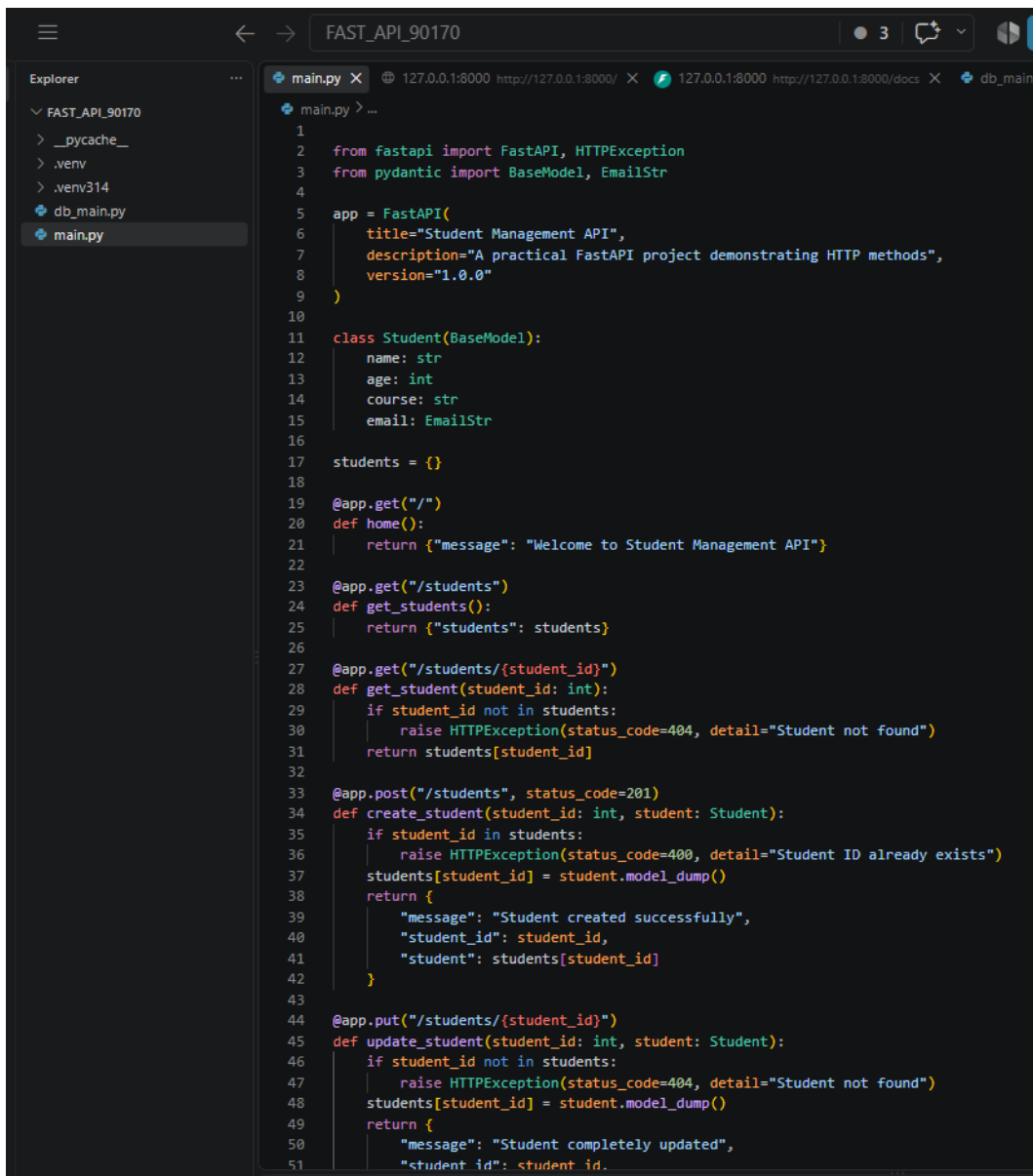
Aim:

To develop a FastAPI-based Student Management API demonstrating HTTP methods (GET, POST, PUT, DELETE) with Pydantic validation and error handling. The API was further integrated with a MySQL database using SQLAlchemy and PyMySQL to perform CRUD operations on persistent data storage.

PART A – FastAPI Student Management / HTTP Methods

1. FastAPI Application

FastAPI is used to create the Student Management API with a Pydantic model for data validation and Uvicorn is used to run the application.



```
1
2 from fastapi import FastAPI, HTTPException
3 from pydantic import BaseModel, EmailStr
4
5 app = FastAPI(
6     title="Student Management API",
7     description="A practical FastAPI project demonstrating HTTP methods",
8     version="1.0.0"
9 )
10
11 class Student(BaseModel):
12     name: str
13     age: int
14     course: str
15     email: EmailStr
16
17 students = {}
18
19 @app.get("/")
20 def home():
21     return {"message": "Welcome to Student Management API"}
22
23 @app.get("/students")
24 def get_students():
25     return {"students": students}
26
27 @app.get("/students/{student_id}")
28 def get_student(student_id: int):
29     if student_id not in students:
30         raise HTTPException(status_code=404, detail="Student not found")
31     return students[student_id]
32
33 @app.post("/students", status_code=201)
34 def create_student(student_id: int, student: Student):
35     if student_id in students:
36         raise HTTPException(status_code=400, detail="Student ID already exists")
37     students[student_id] = student.model_dump()
38     return {
39         "message": "Student created successfully",
40         "student_id": student_id,
41         "student": students[student_id]
42     }
43
44 @app.put("/students/{student_id}")
45 def update_student(student_id: int, student: Student):
46     if student_id not in students:
47         raise HTTPException(status_code=404, detail="Student not found")
48     students[student_id] = student.model_dump()
49     return {
50         "message": "Student completely updated",
51         "student_id": student_id
```

Figure 1: FastAPI Application Code

2. Swagger UI

Swagger UI provides an interactive interface for testing the FastAPI endpoints directly from the browser.

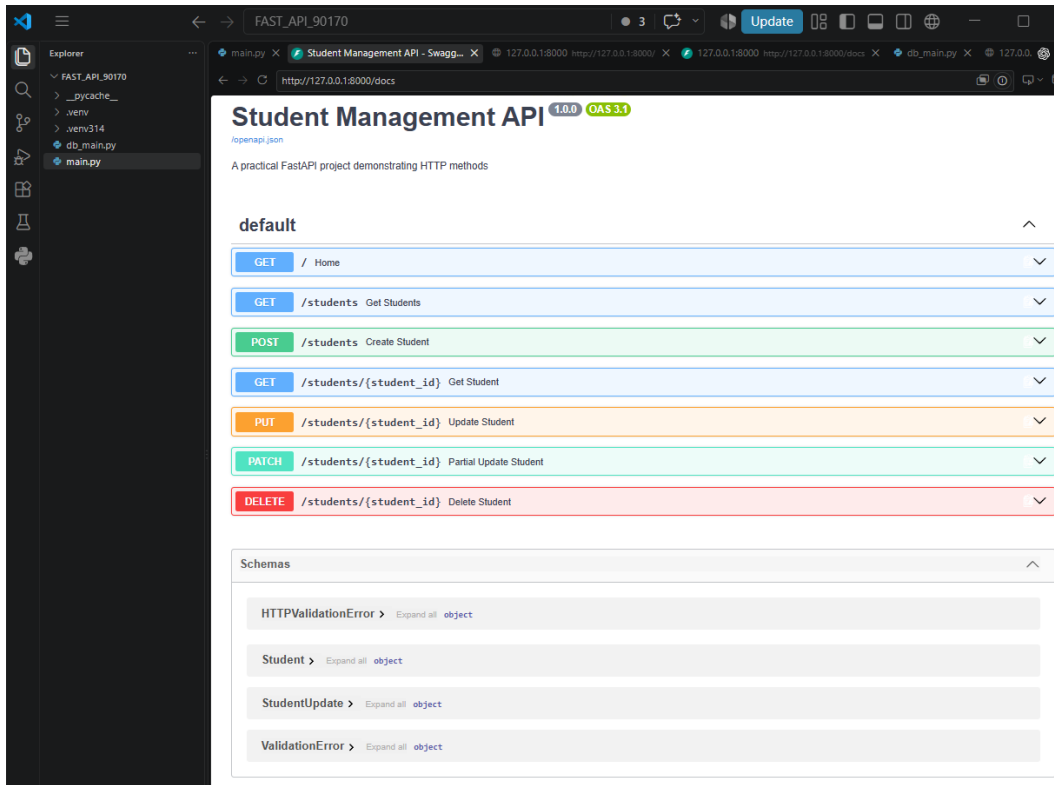


Figure 2: FastAPI Swagger Interface

3. POST Method

The POST method is used to add a new student record through the FastAPI application.

The screenshot displays a REST client interface for a POST request to the endpoint `/students`. The request is titled "Create Student".

Parameters:

Name	Description
<code>student_id</code> *required integer (query)	1

Request body: *required
Type: `application/json`

Edit Value | Schema

```
{
  "name": "sylvia",
  "age": 19,
  "course": "DB",
  "email": "sylvia@gmail.com"
}
```

Execute **Clear**

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/students?student_id=1' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "sylvia",
    "age": 19,
    "course": "DB",
    "email": "sylvia@gmail.com"
  }'
```

Request URL

```
http://127.0.0.1:8000/students?student_id=1
```

Figure 3: POST Student Request and Response

4. GET Method (Retrieve All Students)

The GET method is used to retrieve all student details from the FastAPI application.

GET /students Get Students

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'https://127.0.0.1:8000/students' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/students

Server response

Code

Details

200

Response body

```
{
  "students": {
    "1": {
      "name": "Sylvia",
      "age": 19,
      "course": "DB",
      "email": "Sylvia@gmail.com"
    }
  }
}
```

Download

Response headers

```
content-length: 86
content-type: application/json
date: Thu, 17 Sep 2026 04:54:03 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

Figure 4: GET All Students Request and Response

5. PUT Method

The PUT method is used to completely update an existing student record with new information.

The screenshot shows a REST client interface for a PUT request. The title bar indicates the method is PUT and the endpoint is /students/{student_id} with the description 'Update Student'. There are 'Cancel' and 'Reset' buttons in the top right.

Parameters

Name	Description
student_id ^{required}	
integer	
(path)	

The value for student_id is set to 1.

Request body ^{required}

The content type is set to application/json.

Edit Value | Schema

```
{
  "name": "Sylvia",
  "age": 15,
  "course": "DB",
  "email": "Sylvia@gmail.com"
}
```

Buttons for 'Execute' and 'Clear' are located below the request body.

Responses

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8080/students/1' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Sylvia",
    "age": 15,
    "course": "DB",
    "email": "Sylvia@gmail.com"
  }'
```

Request URL

```
http://127.0.0.1:8080/students/1
```

Server response

Code	Details
------	---------

Figure 5: PUT Student Update Request and Response

6. GET Method (Retrieve Single Student)

The GET method with student_id parameter retrieves a specific student record from the application.

GET
/students/{student_id}
Get Student

Parameters
Cancel

Name	Description
student_id integer (path)	1

Execute
Clear

Responses

Curl

```
curl -X "GET" \
  "http://127.0.0.1:8000/students/1" \
  -H "accept: application/json"
```

Request URL

```
http://127.0.0.1:8000/students/1
```

Server response

Code	Details
200	Response body <pre>{ "name": "Sylvia", "age": 17, "course": "DE", "email": "Sylvia@gmail.com" }</pre> Download Response headers <pre>content-length: 68 content-type: application/json date: Thu, 17 Sep 2026 04:58:57 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response Media type: application/json Controls Accept header. Example Value Schema	No links

Figure 6: GET Single Student Request and Response

7. DELETE Method

The DELETE method is used to remove a student record from the FastAPI application.

DELETE /students/{student_id} Delete Student

Parameters

Name	Description
student_id required	
integer	
(path)	

1

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \
'http://127.0.0.1:8000/students/1' \
-H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/students/1

Server response

Code	Details
200	Response body <pre>{ "message": "Student deleted successfully", "student_id": 1, "student": { "name": "Sylvia", "age": 10, "course": "DB", "email": "Sylvia@gmail.com" } }</pre> Download

Response headers

```
content-length: 136
content-type: application/json
date: Thu, 17 Sep 2026 04:59:31 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value Schema

Figure 7: DELETE Student Request and Response

8. Error Handling

Error handling is implemented to return appropriate HTTP status codes and error messages when invalid requests are made, such as attempting to retrieve a non-existent student record.

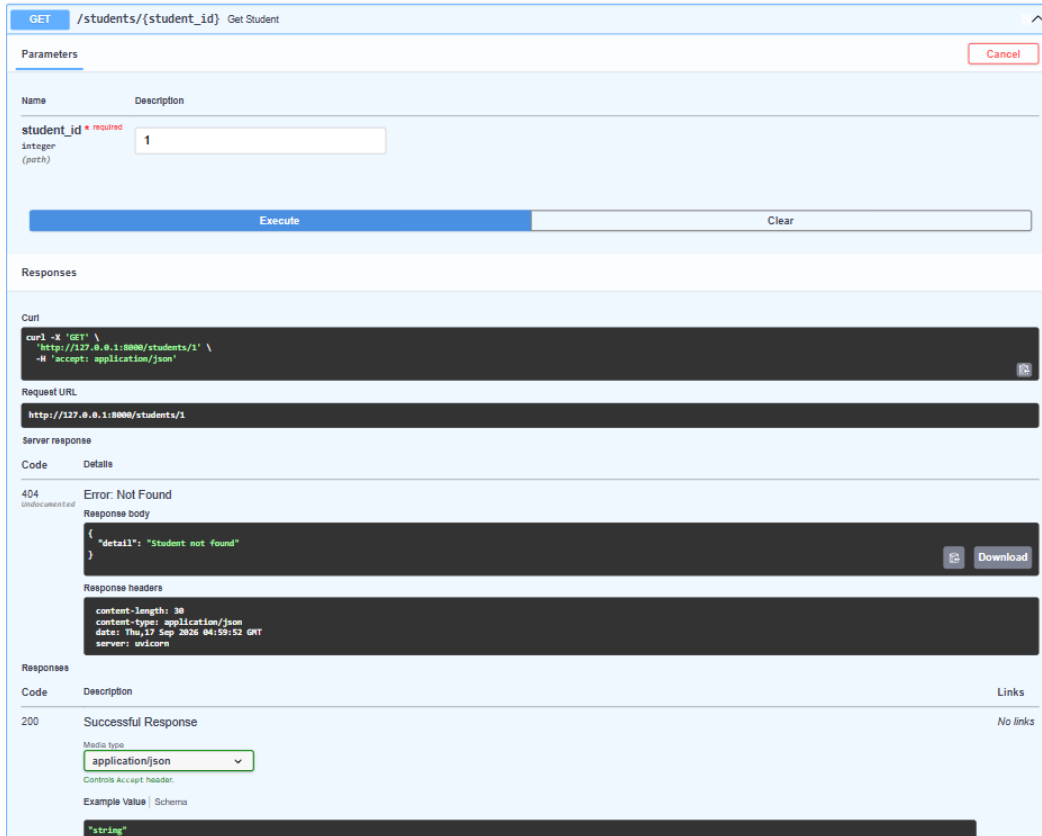


Figure 8: Error Handling (404 Not Found Response)

PART B – FastAPI with MySQL Database

1. FastAPI-MySQL Connection

FastAPI was connected to the MySQL database using SQLAlchemy ORM and PyMySQL driver to enable persistent data storage.



≡

← →

FAST_API_90170

Explorer

FAST_API_90170

- __pycache__
- .venv
- .venv314
- db_main.py
- main.py

main.py × Student Management API - Swagg... × 127.0.0.1:8000 http://127.0.0.1

db_main.py > ...

```
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3 from sqlalchemy import create_engine, text
4 from sqlalchemy.engine import URL
5
6 app = FastAPI(
7     title="Student Database API - 90170",
8     description="FastAPI connection with MySQL",
9     version="1.0"
10 )
11
12 DATABASE_URL = URL.create(
13     drivername="mysql+pymysql",
14     username="root",
15     password="",
16     host="localhost",
17     port=3306,
18     database="university_90170"
19 )
20
21 engine = create_engine(DATABASE_URL)
22
23
24 class Student(BaseModel):
25     id: int
26     name: str
27     department: str
28
29
30 @app.get("/")
31 def home():
32     return {"message": "FastAPI connected project - 90170"}
33
34
35 @app.get("/students")
36 def get_students():
37     with engine.connect() as connection:
38         result = connection.execute(
39             text("SELECT * FROM students")
40         )
41         students = result.mappings().all()
42         return [dict(student) for student in students]
43
44
45 @app.post("/students")
46 def create_student(student: Student):
47     with engine.begin() as connection:
48         connection.execute(
49             text("""
50                 INSERT INTO students (id, name, department)
51                 VALUES (:id, :name, :department)
52             """),
53             {
54                 "id": student.id,
55                 "name": student.name,
56                 "department": student.department
57             }
58         )
59
60     return {
61         "message": "Student created",
62         "student": student
63     }
```

Figure 9: FastAPI Connection Code with MySQL

2. Verify Data in MySQL Workbench

The data inserted through FastAPI was verified directly in MySQL Workbench using SELECT * FROM students query on the university_90170 database.

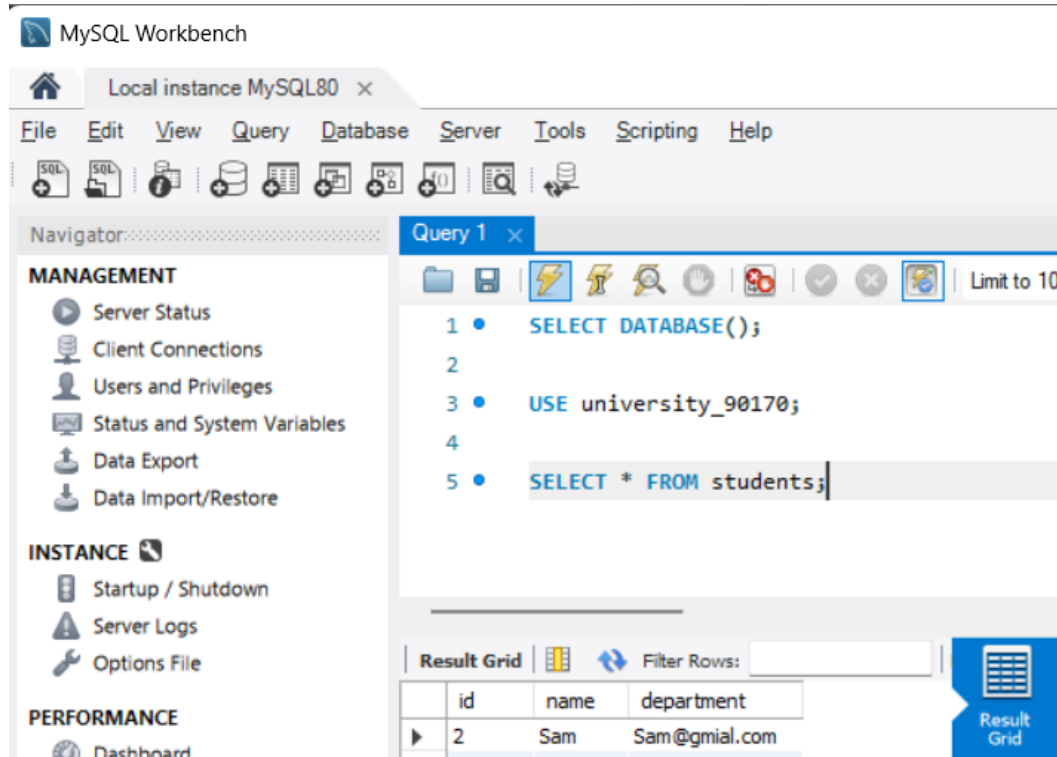


Figure 10: Student Records Verified in MySQL Workbench

Result:

The FastAPI operations were executed successfully and the connection between FastAPI and MySQL was verified.